

> TDA Lista doblemente ligada

Una lista doblemente ligada es una lista lineal donde cada nodo lleva consigo dos enlaces, uno al nodo siguiente y otro al nodo anterior. Debido a esto último, este tipo de lista no necesitan un nodo especial para acceder a ellas ya que puede recorrerse en ambos sentidos a partir de cualquier nodo.

• Implementación en C++

```
class Nodo {
private:
    int info;
    Nodo* sig;
    Nodo* ant;
public:
    Nodo();
    Nodo(int info); // Sobrecarga de constructores
    Nodo(int info, Nodo* sig);
    Nodo(int info, Nodo* ant);
    Nodo(int info, Nodo* sig, Nodo* ant);
    ~Nodo();
};
```

Diferencias en la clase Nodo entre los tipos de listas

	Simp. ligada	Doblem. ligada
Atributos	- info: cualq. tipo de dato - sig: apunt. al sig nodo	- info: cualq. tipo de dato - sig: dir. al sig nodo - ant: dir. al ant nodo
Métodos	Sobre carga en construct. uno donde solo se ingrese info y otro donde se agreguen ambos atributos.	Sobre carga de constructores igual a la simple más dos donde le pases ant. y, sig y ant.

- Operaciones de una TDA doblemente ligada:

1. **Crear lista:** Inicializa la lista doblemente enlazada, donde además puedes crear un acceso directo a la cola, así como se tenía antes de cabeza. Permite acceder desde ambos extremos.

2. **Insertar al inicio:** Crea el nodo y le da la dirección a la cabeza y en caso de ser el primero Camb. a la cola. Las direc. del nodo se asignan respectivamente.

3. **Insertar al final:** Al tener la dirección de la cola, es más fácil insertar al final. Al nuevo nodo en su parte anterior le asigna su direc. la cola y su parte sig. es nula.

4. **Eliminar al inicio:** Se borra el primer nodo sin antes asignar el segundo a la cabeza, ahora a este segundo su parte ant. apunta a null.

5. **Eliminar del final:** Se borra la cola sin antes asignar su parte ant. a la cola. Además, la nueva cola en su parte sig apunta a null.

6. **Buscar elemento:** Funciona igual que en las listas simplemente ligadas, pero ahora se puede optimizar la función si la posición de donde está la información es mayor que la mitad. De esta manera podemos buscar desde la cola.

7. **Recorrer la lista:** Funciona igual que mostrar todo de las listas simples, pero ahora es posible de recorrerla en reversa.

8. **Obtener tamaño:** Recorre la lista contando cada desplazamiento para así obtener el tamaño total de la lista.

9. **Verificar Vacía:** Checa si la cabeza o la cola está vacía.

10. **Eliminar todo:** Elimina toda la lista y libera la memoria.

Nodo

- info: int
- sig: Nodo*
- ant: Nodo*
+ Nodo
+ Nodo(info: int)
+ Nodo(info: int, sig: Nodo*)
+ Nodo(info: int, ant: Nodo*)
+ Nodo(info: int, sig: Nodo*, ant: Nodo*)
+ ~Nodo

Lista

- info: int
- posicion: int
- cabeza: Nodo*
- cola: Nodo*
+ Insert_inicio(info: int): void
+ Insert_final(info: int): void
+ Insert_posicion(info: int, posicion: int): void
+ Elim_inicio(): void
+ Elim_final(): void
+ Elim_posicion(): void
+ Buscar(info: int, posicion: int, metodo: int): void
+ Mostrar(): void
+ Tamaño(): void
+ Vacía(): bool
+ Eliminar_todo(): void

